

A proof of $P \neq NP$ (New symmetric encryption algorithm against any linear attacks and differential attacks)

Gao Ming*

March 5, 2022

Abstract

P vs NP problem is the most important unresolved problem in the field of computational complexity. Its impact has penetrated into all aspects of algorithm design, especially in the field of cryptography. The security of cryptographic algorithms based on short keys depends on whether P is equal to NP. In fact, the security requirements for cryptographic keys are much stricter than those for $P \neq NP$, the security of the key must ensure not only a sufficiently high computational complexity to crack it, but also consider the security of each bit of the key, while fully avoiding the effectiveness of various attack methods.

In this paper, we innovatively propose a new encoding mechanism and develop a novel block symmetric encryption algorithm, whose encryption and decryption can be completed in linear time. For the attacker, in the case where only the plaintext-ciphertext correspondence is known, the problem of cracking the key is equivalent to solving a system of equations which contains at least one variable that cannot be solved, and the number of possible values for one variable is exponentially to the length of the key. To solve this system of equations, it is necessary to exhaustively search for at least one variable, thus proving that the computational complexity of cracking the key is exponential. So the decryption is a one-way function, and according to “the existence of one-way function means $P \neq NP$ ”, thus solving the unsolved problem of P vs NP.

In addition, this paper delves into the underlying mathematical laws of this new encoding mechanism, and develops a right multiplication operation to binary. Based on this right multiplication operation, we further constructed a nonlinear operation and designed another block symmetric encryption algorithm that is resistant to all forms of linear and differential attacks.

*20070602094@alu.cqu.edu.cn

1 Introduction

Cryptography is one of the most important applications in the field of communication and computer science. In recent years, with the application of commerce, enterprises, banks and other departments, cryptography has been developed rapidly. Especially after Shannon put forward the mathematical analysis of security in “Communication theory of secrecy systems”[1], various design tools for cipher algorithms and corresponding attack tools have been developed one after another.

The P vs NP problem was first proposed by Cook[10] and has become one of the most important unsolved problems in the field of computer science. At the same time, the P vs NP problem has also become the most important theoretical foundation in the field of cryptography, determining the research direction of cryptography. $P \neq NP$ is the most basic assumption in cryptography, but this is only a necessary condition. In fact, the security requirements of a key are much stricter than $P \neq NP$. This is because even if $P \neq NP$, there exists an encryption algorithm, the computational complexity of cracking the key is exponential to the length of the key, which does not guarantee that each bit of the key is secure. To ensure that each bit of the key is secure, it is necessary to ensure that exhaustive search is only effective for cracking the key, which is much stricter than the exponential computational complexity to cracking the key.

Among various attack methods, linear attack and differential attack are the two most core attack methods, which have become important analytical tools in the field of cryptography, driving the improvement of encryption algorithm.

Linear attack was first proposed by M. Matsui[2], this is an attack method that is currently applicable to almost all block encryption algorithms. Kaliski BS[3] proposed a multi-linear attack based on the linear attack, but the multi-linear attack has many limitations. And the Biryukov A[4] and Chao, J.Y[5] and others further improved the framework of multi-linear attacks, thus making linear attacks a larger application.

The differential attack method was first proposed by Eli Biham[6]. Biham E[7] extended it to a more powerful attack method. Tsunoo[8] further constructed multiple attack methods. These attack methods have extremely high skill in the attack process, which is worthy in-depth study.

In Chapter 3, we propose a novel encoding mechanism which we named Eagle. The distinguishing feature of Eagle encoding algorithm from any other encoding algorithm is that for any given group of input and output texts, the key parameters are unsolvable.

In Chapter 4, we further delve into the mathematical laws behind the Eagle encoding algorithm and discover an operation that satisfies the allocation law for XOR operations. This operation is very similar to the allocation law of multiplication operations, but does not satisfy the commutative and associative laws, we refer to it as a “right multiplication operation on binary”. Then we construct a binary operation of two-dimensional vectors which serves as a key ingredient for the subsequent design of encryption algorithms.

In Chapter 5, we constructed a block symmetric encryption algorithm based on the Eagle encoding algorithm which we named it Eagle. In this encryption algorithm, there are three completely independent sets of random keys, each containing two random numbers. For the plaintexts with K groups, $6K + 6$ random numbers will be randomly generated at the encryption party. After encryption is completed, ciphertext text with $7K + 6$ groups will be generated. After encryption is completed, the encryption party will delete these generated intermediate random numbers. For the attacker, knowing the plaintext-ciphertext correspondence, the problem of cracking the key is equivalent to solving a series of equations which contain at least one variable that need to be exhaustively searched, thus conclude that the computational complexity of cracking the key is exponential. On the other hand, when knowing the ciphertext, the computation for decrypting with a given key is linear to the length of the key. This also means that decryption is a one-way function, thus solving the P vs NP problem based on the conclusion that “one-way functions exist means $P \neq NP$ ”.

In fact the encryption algorithm developed in Chapter 5 does not meet the security requirements for the key. This is because, when knowing the plaintext-ciphertext correspondence, although the complexity of cracking the key is exponential to the length of the key, it does not guarantee that only exhaustive search is effective for cracking the key. There may be some other attack methods that can effectively reduce the difficulty of cracking the key. Such as for a 256 bit key, although it can ensure that the computational complexity of cracking the key is at least $O(2^{64})$, this is still far from the ideal security of $O(2^{256})$. The encryption algorithm constructed in Chapter 5 only analyzes the computational complexity of cracking the key given a plaintext ciphertext correspondence, which does not guarantee its ability to resist linear and differential attacks. For attackers, they can construct plaintexts with certain characteristics multiple times, then find some structures that we did not anticipate from the relationship between plaintext and ciphertext.

In Chapter 6, we first construct nonlinear operations of one parameter on another, such as “high-order” operations. Based on this nonlinear construction, we redesigned a block symmetric encryption algorithm and named it *Eagle**. For *Eagle**, knowing any plaintext ciphertext correspondence, there is no more effective way for attackers to crack the key than exhaustive search.

On the other hand, for the *Eagle** encryption algorithm, the key is divided into two independent parts, and the ciphertext can be seen as three independent parts. Two parts of the ciphertext is completely determined by the intermediate random number which is completely unrelated to the plaintext. The third part of the ciphertext is determined by the intermediate random number together with the plaintext sequence, but it is completely indistinguishable whether the uncertainty of the ciphertext brought by the random number or brought by different plaintexts. This theoretically ensure that *Eagle** can resist all forms of linear attacks and differential attacks.

2 Notations

Some notations

$\oplus$ - XOR operation.

D^{+n} - Do left cycle shift of D by n bits.

$m[i]$ - The i -th bit of m .

$E_{w_0, w_1}(m)$ - Excute Eagle encoding algorithm as shown in Algorithm 1.

$D_{w_0, w_1}(s)$ - Excute Eagle decoding algorithm as shown in Algorithm 2.

$m * \delta$ - Excute the right multiplication of m with δ as shown in Algorithm 3.

s/δ - Excute the right division of s to δ as shown in Algorithm 4.

$m * \delta^2$ - Excute the right multiplication of m with δ twice as $m * \delta * \delta$ as shown in Algorithm 7.

s/δ^2 - Excute the right division of s to δ twice as $s/\delta/\delta$ as shown in Algorithm 8.

$(m_1, m_2) \otimes (k_1, k_2) = (c_1, c_2)$ - A binary operation of two-dimensional vectors defined in chapter 4.2.

3 Introduction to Eagle encoding algorithm

We first introduce two common bit operations. XOR denoted as $\oplus$. Do left cycle shift of D by n bits which can be denoted as D^{+n} , for example $(10011010)^{+2} = (01101010)$.

We select two L -bits parameters w_0 and w_1 , have odd number of different bits. For example $w_0 = 10010011$ and $w_1 = 11000111$ have 3 bits (3 is odd) different.

$$w_0 = 1\textcolor{red}{00}1\textcolor{red}{00}11$$

$$w_1 = 1\textcolor{red}{1}0\textcolor{red}{00}1\textcolor{red}{11}$$

We set the initial state of L -bit as s_0 , we choose one parameter $w \in \{w_0, w_1\}$, without loss of generality, assume that we choose $w = w_0$, then we define the following calculation

$$s_1 = w \oplus (s_0 \oplus s_0^{+1}) = w_0 \oplus (s_0 \oplus s_0^{+1}) \quad (3.1)$$

From (3.1), we can easily know

$$s_0 \oplus s_0^{+1} = s_1 \oplus w \quad (3.2)$$

If we only know s_1 but not s_0 , we don't know whether $w = w_0$ or $w = w_1$ we used in (3.1), we can check it through a simple trial-and-error test. For example, we guess the parameter $w = w_1$ was used in (3.1), we need to find s_x to satisfy

$$s_x \oplus s_x^{+1} = s_1 \oplus w_1 \quad (3.3)$$

In fact, since w_0 and w_1 have odd number of different bits, such s_x does not exist. See Theorem 1 for details.

[Theorem 1] Two L -bit parameters w_0 and w_1 which have odd number of different bits, given s_0 , we set $s_1 = w_0 \oplus (s_0 \oplus s_0^{+1})$, then there doesn't exist s_x satisfy $s_x \oplus s_x^{+1} = s_1 \oplus w_1$.

Proof. Firstly, by definition we have

$$s_1 \oplus w_1 = w_0 \oplus (s_0 \oplus s_0^{+1}) \oplus w_1 = (s_0 \oplus s_0^{+1}) \oplus (w_0 \oplus w_1) \quad (3.4)$$

Where w_0 and w_1 have odd number of different bits, so there are odd number of 1 in the bit string of $w_0 \oplus w_1$.

Proof by contradiction, we suppose that there exists s_x satisfy $s_x \oplus s_x^{+1} = s_1 \oplus w_1$, then by (3.4), we have

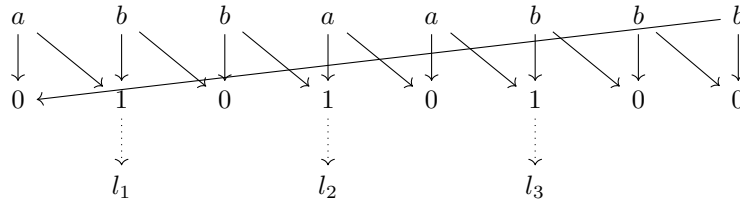
$$s_x \oplus s_x^{+1} = (s_0 \oplus s_0^{+1}) \oplus (w_0 \oplus w_1) \quad (3.5)$$

By simple calculation we have

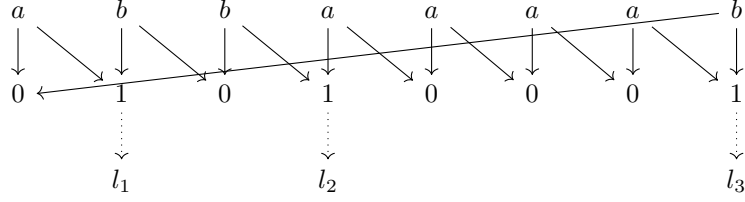
$$(s_0 \oplus s_x) \oplus (s_0 \oplus s_x)^{+1} = w_0 \oplus w_1 \quad (3.6)$$

We set $s_y = s_0 \oplus s_x$, then there are odd number of 1 in the bit string of $s_y \oplus s_y^{+1}$, without of generality, we suppose that the bits with 1 are $l_1, l_2, \dots, l_u$ (u is odd). Compare to the first bit of s_y , the $l_1 + 1$ bit of s_y is different from the first bit of s_y , the $l_2 + 1$ bit of s_y is the same with the first bit of s_y , the $l_3 + 1$ bit of s_y is different from the first bit of s_y , the $l_4 + 1$ bit of s_y is the same with the first bit of s_y , and so on, since u is odd, $u - 1$ is even, the $l_{u-1} + 1$ bit of s_y is the same with the first bit of s_y .

If $l_u < L$, the $l_u + 1$ bit of s_y is different from the first bit of s_y , so the last bit of s_y is different from the first bit of s_y . On the other hand, the last bit of $s_y \oplus s_y^{+1}$ is 0, so the last bit of s_y is the same with the first bit of s_y . This is contradictory.



If $l_u = L$, the last bit of s_y is different from the first bit of s_y . On the other hand, the last bit of s_y is the same with l_{u-1} bit of s_y which is the same with the first bit of s_y . This is contradictory.



So there doesn't exist s_x satisfy $s_x \oplus s_x^{+1} = s_1 \oplus w_1$. $\square$

Let's go back to the discussion just now, after a trial-and-error test, we can accurately check which one ($w = w_0$ or $w = w_1$) we just used in (3.1).

Now we suppose that there is a binary sequence $m = b_1 b_2 \dots b_L$ with length L . Start with s_0 , read each bit of M from left to right sequentially, when the bit $b[i]$ ($1 \leq i \leq L$) is 0, we set $s_i = w_0 \oplus (s_{i-1} \oplus s_{i-1}^{+1})$, when the bit $b[i]$ is 1, we set $s_i = w_1 \oplus (s_{i-1} \oplus s_{i-1}^{+1})$.

According to the above calculation, for every s_i , we can find the only $w_x = w_0$ or $w_x = w_1$ such that there exists s_{i-1} satisfy $s_{i-1} \oplus s_{i-1}^{+1} = s_i \oplus w_x$. According to the properties of XOR and cyclic shift, we can easily see that there are only two s_{i-1} that satisfy $s_{i-1} \oplus s_{i-1}^{+1} = s_i \oplus w_x$, and the two s_{i-1} with each bit different. As long as we know any one bit of s_{i-1} , s_{i-1} can be uniquely determined. So we only need to save one bit of s_i , finally by s_L , we can completely restore the original state s_0 and the binary sequence m .

Based on the above discussion, we can construct a complete Eagle encoding and decoding algorithm. The entire algorithm consists of three processes: generating parameters, encoding, and decoding.

[Parameter generation]

Firstly we choose two L -bit parameters w_0 and w_1 which have odd number of different bits, then we choose L -bit initial state s_0 .

[Encoding]

For input data m , we record $m[i]$ ($1 \leq i \leq L$) as the i -th bit of m , $m[i] \in \{0, 1\}$, L is the length of m , the encoding process is as follows.

Algorithm 1 *Eagle*-Encoding.

Input:

Parameter: (w_0, w_1) .
Input: $m = [m[1], m[2], \dots, m[L]]$.
Random number: s_0 .

Output:

Output: (c, s_L) ,
1: Define c as a bit array with length L .
2: **for** $i = 1; i \leq L; i++$ **do**
3: $c[i] \leftarrow s_{i-1}[L]$
4: **if** $m[i] = 0$ **then**
5: $s_i \leftarrow w_0 \oplus (s_{i-1} \oplus s_{i-1}^{+1})$
6: **else**
7: $s_i \leftarrow w_1 \oplus (s_{i-1} \oplus s_{i-1}^{+1})$
8: **end if**
9: **end for**

Where the i -th bit of c takes the last bit of state s_{i-1} .

[Decoding]

The output (c, s_L) of the above encoding process is used as the input of the decoding process, the decoding process is as follows.

Algorithm 2 *Eagle*-Decoding.

Input:

Parameter: (w_0, w_1) .
Input: $c = [c[1], c[2], \dots, c[L]]$, s_L .

Output:

Output: m .
1: **for** $i = L; i \geq 1; i--$ **do**
2: Define x as a bit. ($x = 0$ or $x = 1$)
3: Do trial-and-error testing with $s_i \oplus w_0$ or $s_i \oplus w_1$, find the unique w_x ($x = 0$ or $x = 1$) satisfy $s_x \oplus s_x^{+1} = s_i \oplus w_x$.
4: $m[i] \leftarrow x$
5: For the two possible s_x satisfy $s_x \oplus s_x^{+1} = s_i \oplus w_x$, we select the one which the last bit is equal to $c[i]$ as s_{i-1} .
6: **end for**

Now we give an example of the above algorithms.

[Example 1] Eagle encoding

We choose the parameters as $w_0 = 10010011$, $w_1 = 11000111$, $s_0 = 01011001$, $m = 10010101$.

Since $m[1] = 1$, we set $w = w_1$, then we have

$$s_1 = w \oplus (s_0 \oplus s_0^{+1}) = 11000111 \oplus 11101011 = 00101100$$

Since $m[2] = 0$, we set $w = w_0$, then we have

$$s_2 = w \oplus (s_1 \oplus s_1^{+1}) = 10010011 \oplus 01110100 = 11100111$$

Since $m[3] = 0$, we set $w = w_0$, then we have

$$s_3 = w \oplus (s_2 \oplus s_2^{+1}) = 10010011 \oplus 00101000 = 10111011$$

Since $m[4] = 1$, we set $w = w_1$, then we have

$$s_4 = w \oplus (s_3 \oplus s_3^{+1}) = 11000111 \oplus 11001100 = 00001011$$

Since $m[5] = 0$, we set $w = w_0$, then we have

$$s_5 = w \oplus (s_4 \oplus s_4^{+1}) = 10010011 \oplus 00011101 = 10001110$$

Since $m[6] = 1$, we set $w = w_1$, then we have

$$s_6 = w \oplus (s_5 \oplus s_5^{+1}) = 11000111 \oplus 10010011 = 01010100$$

Since $m[7] = 0$, we set $w = w_0$, then we have

$$s_7 = w \oplus (s_6 \oplus s_6^{+1}) = 10010011 \oplus 11111100 = 01101111$$

Since $m[8] = 1$, we set $w = w_1$, then we have

$$s_8 = w \oplus (s_7 \oplus s_7^{+1}) = 11000111 \oplus 10110001 = 01110110$$

Take the last bit of $s_0, s_1, s_2, s_3, \dots, s_7$ as $c = 10111001$.

We use $(c, s_8) = (10111001, 01110110)$ as the results generated by Eagle encoding.

Next, we will provide the example of Eagle decoding.

[Example 2] Eagle decoding

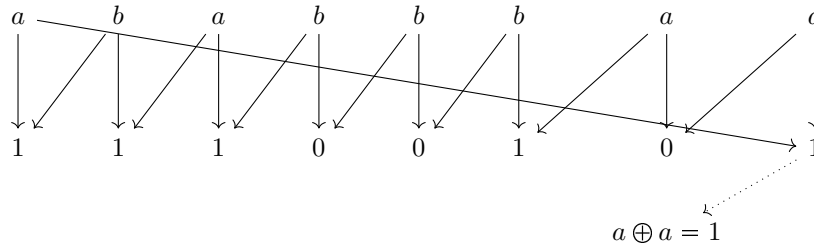
Known $w_0 = 10010011$, $w_1 = 11000111$, $s_8 = 01110110$, $c = 10111001$, how to compute m ?

Firstly by $s_8 = w \oplus (s_7 \oplus s_7^{+1}) \Rightarrow s_7 \oplus s_7^{+1} = s_8 \oplus w$, we don't know whether $w = w_0$ (the 8-th bit of m is 0) or $w = w_1$ (the 8-th bit of m is 1), we need a trial-and-error test.

Let's assume $w = w_0$, we have

$$s_7 \oplus s_7^{+1} = s_8 \oplus w_0 = 01110110 \oplus 10010011 = 11100101$$

There is no such s_7 satisfies $s_7 \oplus s_7^{+1} = 11100101$.



Let's assume $w = w_1$, we have

$$s_7 \oplus s_7^{+1} = s_8 \oplus w_1 = 01110110 \oplus 11000111 = 10110001$$

There exists two solutions $s_7 = 01101111$ and $s_7 = 10010000$, these two solutions are bit reversed.

Since the 8-th bit of c is $c[8] = 1$, the last bit of s_7 is 1, we have $s_7 = 01101111$.

According to the above, because $w = w_1$, the 8-bit of m is 1.

Similarly we get $m = 10010101$.

It is not difficult to find that the above encoding process and decoding process are correct, that is (c, s_L) generated by encoding from m can be completely restored through the decoding process. In addition, the encoding process is sequential encoding in the order of m 's bits, and the decoding process is sequential decoding in the reverse order of c 's bits.

$$s_0 \xrightarrow{m[1]} s_1 \xrightarrow{m[2]} s_2 \xrightarrow{m[3]} \dots \xrightarrow{m[L]} s_L$$

$$s_0 \xleftarrow{c[1]} s_1 \xleftarrow{c[2]} s_2 \xleftarrow{c[3]} \dots \xleftarrow{c[L]} s_L$$

We also noticed the fact that in the above encoding and decoding process, all inputs and outputs do not need to appear s_0 , This means that the selection of s_0 will not affect the correctness of the encoding process and decoding process. The arbitrariness of s_0 will bring the unpredictability of the encoded output.

For the convenience of the discussion in the following chapters, here we briefly analyze the effect of unpredictability of s_0 on the encoded output.

Given the parameters w_0 and w_1 that have odd number of different bits, for a certain input m of L bits, since s_0 is arbitrarily selected, it is obvious that c is uncertain, but is the final state s_L necessarily uncertain?

In fact, the answer is no. In some cases, such as $L = 2^u$ (that is, the parameter length is the power of 2), the final state s_L is determined for different choices of s_0 . The final state variable s_L which is the output of the encoding process is only related to the input m and has nothing to do with the choice of the initial state s_0 . See Theorem 2 for details.

[Theorem 2] In Eagle encoding algorithm, given the parameters w_0 and w_1 that have L bits with odd bits different, for a certain L bit input m , if $L = 2^u$ is satisfied, then for any initial state s_0 , after the Eagle encoding process, the final state s_L is only related to the input m , and is unrelated with the choice of the initial state s_0 .

Proof. We represent m as binary stream $x_1x_2\dots x_L$, which $x_i \in \{0, 1\}$, $1 \leq i \leq L$.

We execute the Eagle encoding process to m from x_1 to x_L as follows.

$$s_1 = w_{x_1} \oplus (s_0 \oplus s_0^{+1}) = f_1(w_{x_1}) \oplus (s_0 \oplus s_0^{+1})$$

$$\begin{aligned}
s_2 &= w_{x_2} \oplus (s_1 \oplus s_1^{+1}) = f_2(w_{x_1}, w_{x_2}) \oplus (s_0 \oplus s_0^{+2}) \\
s_3 &= w_{x_3} \oplus (s_2 \oplus s_2^{+1}) = f_3(w_{x_1}, w_{x_2}, w_{x_3}) \oplus (s_0 \oplus s_0^{+1} \oplus s_0^{+2} \oplus s_0^{+3}) \\
s_4 &= w_{x_4} \oplus (s_3 \oplus s_3^{+1}) = f_4(w_{x_1}, w_{x_2}, w_{x_3}, w_{x_4}) \oplus (s_0 \oplus s_0^{+4})
\end{aligned}$$

It is not difficult to find that for any $m = 2^v$, $s_m = f_m(w_{x_1}, \dots, w_{x_m}) \oplus (s_0 \oplus s_0^{+m})$ holds, this can be proved by a simple mathematical induction.

In fact, the conclusion is correct for $v = 1$.

We assume that the conclusion is correct for $v - 1$, we have

$$s_{m/2} = f_{m/2}(w_{x_1}, \dots, w_{x_{m/2}}) \oplus (s_0 \oplus s_0^{+m/2})$$

Since $s_{m/2}$ to s_m must do calculations with $m/2$ steps, we have

$$\begin{aligned}
s_m &= f_m(w_{x_1}, \dots, w_{x_m}) \oplus (s_0 \oplus s_0^{+m/2}) \oplus (s_0^{+m/2} \oplus s_0^{+m}) \\
&= f_m(w_{x_1}, \dots, w_{x_m}) \oplus (s_0 \oplus s_0^{+m})
\end{aligned}$$

Since $L = 2^u$, we have $s_L = f_L(w_{x_1}, \dots, w_{x_L}) \oplus (s_0 \oplus s_0^{+L})$, where $f_i(\dots)$ is irrelevant with s_0 , by definition of cycle shift, we have $s_0 = s_0^{+L}$, so $s_L = f_L(w_{x_1}, \dots, w_{x_L})$ which is irrelevant with s_0 .

□

From theorem 2, for any parameters w_0 and w_1 with length $L = 2^u$, for any initial state s_0 , execute Eagle encoding on m to obtain s_L which is irrelevant with s_0 .

Next we introduce two symbols $\mathbb{E}$ and $\mathbb{D}$.

$\mathbb{E}_{w_0, w_1}(m) = s_L$: use the key (w_0, w_1) to execute Eagle encoding algorithm (Algorithm 1) on any initial state s_0 and the given input m to obtain c and s_L , which s_0 and c will not be analyzed anymore.

$\mathbb{D}_{w_0, w_1}(s_L) = m$: use the key (w_0, w_1) to execute Eagle decoding algorithm (Algorithm 2) on any c and the given s_L to obtain s_0 and m .

$\mathbb{E}$ and $\mathbb{D}$ both represent a complete Eagle encoding process and Eagle decoding process, and their introduction is mainly for the convenience of deriving encryption algorithms later.

In all the following chapters of this paper, we assume the length L is a power of 2.

4 In depth analysis of Eagle encoding algorithm

Back to the Eagle encoding algorithm, we noticed that w_0 differs from w_1 with an odd number of bits, which is equivalent to the binary representation of $w_0 \oplus w_1$ containing an odd number of 1. There are 2^{2L-1} pairs of (w_0, w_1) that satisfy an odd number of different bits. This is because among all (w_0, w_1) , w_0 and w_1 contain the same number of odd bits different as even bits different.

Next, let's take a closer look at the features of the Eagle encoding algorithm.

For the parameter (w_0, w_1) , we have another completely equivalent representation (δ, w_0) , where $\delta = w_0 \oplus w_1$, which means that (w_0, w_1) and (δ, w_0) can be equivalently transformed. Since the selection of s_0 is independent of s_L , without loss of generality, we set $s_0 = 0$, when given (δ, w_0) , we only observe the relationship between the final state s_L and the input $m = b_1 b_2 \dots b_L$, where $b_i \in \{0, 1\}$. We excute the Eagle encoding algorithm with the first bit to obtain

$$s_1 = w_{b_1} \oplus (s_0 \oplus s_0^{+1}) = w_0 \oplus g_1(b_1, \delta)$$

Where $g_1(b_1, \delta) = 0$ when $b_1 = 0$ and $g_1(b_1, \delta) = \delta$ when $b_1 = 1$. We excute the Eagle encoding algorithm with the second bit to obtain

$$\begin{aligned} s_2 &= w_{b_2} \oplus (s_1 \oplus s_1^{+1}) \\ &= (w_0 \oplus (w_0 \oplus w_0^{+1})) \oplus g_2(b_1, b_2, \delta) \\ &= w_0^{+1} \oplus g_2(b_1, b_2, \delta) \end{aligned}$$

Where $g_2(b_1, b_2, \delta)$ is a function about δ when b_1, b_2 is given. We excute the Eagle encoding algorithm with the third bit to obtain

$$\begin{aligned} s_3 &= w_{b_3} \oplus (s_2 \oplus s_2^{+1}) \\ &= (w_0 \oplus w_0^{+1} \oplus w_0^{+2}) \oplus g_3(b_1, b_2, b_3, \delta) \end{aligned}$$

So far we see that $s_i = h_i(w_0) \oplus g_i(b_1, b_2, \dots, b_i, \delta)$, where $h_i(w_0)$ can be regarded as the state obtained by executing i bits Eagle encoding algorithm when $m = 0$, $g_i(b_1, b_2, \dots, b_i, \delta)$ is a function about δ when $b_1, b_2, \dots, b_i$ are given.

When $L = 2^u$, we set $m = 0, s_0 = 0$, after executing Eagle encoding, the final state is $s_L = w_0^{+(L-1)}$. When $m \neq 0$, after excuting Eagle encoding, the final state is

$$s_L = w_0^{+(L-1)} \oplus g_L(b_1, b_2, \dots, b_L, \delta) \quad (4.1)$$

Where $g_L(b_1, b_2, \dots, b_L, \delta)$ can also be expressed as $g_L(m, \delta)$. It is not difficult to see that $g_L(m, \delta)$ is a series of XOR operations with δ^{+k} given m , this is also equivalent to $g_L(m, \delta) = c_1 \delta^{+1} \oplus c_2 \delta^{+2} \oplus \dots \oplus c_L \delta^{+L}$, where $c_i \in \{0, 1\}$ and $(c_1, c_2, \dots, c_L)$ only determined by m . So (4.1) can ultimately be expressed as

$$s_L = w_0^{+(L-1)} \oplus g_L(m, \delta) = w_0^{+(L-1)} \oplus (c_1 \delta^{+1} \oplus c_2 \delta^{+2} \oplus \dots \oplus c_L \delta^{+L}) \quad (4.2)$$

Furthermore, we observe several important facts, as shown in the following theorems.

[Theroem 3] $\forall m, s$, there exists 2^{L-1} different groups (w_0, w_1) satisfy that $\mathbb{E}_{w_0, w_1}(m) = s$. For any δ , there exists a unique w_0 that $\mathbb{E}_{w_0, \delta \oplus w_0}(m) = s$.

Proof. From (4.2), we know $\mathbb{E}_{w_0, w_1}(m) = w_0^{+(L-1)} \oplus g_L(m, \delta)$, through a simple transformation we obtain $w_0^{+(L-1)} = g_L(m, \delta) \oplus s$, it is obvious that for any δ, m, s , the right side of the equation is a difinite result, so there exists a unique

w_0 valid.

Additionally, due to the odd number of bits differ between w_0 and w_1 , obviously $\delta = w_0 \oplus w_1$ have an odd number of bits that are 1, that is to say, there are 2^{L-1} different δ . $\square$

The conclusion of Theorem 3 indicates that, for any $\mathbb{E}_{w_0, w_1}(m) = s$, when only given (m, s) , there are exponentially different (w_0, w_1) valid.

[Theorem 4] For any m_1, s_1, m_2, s_2 , when the parity between m_1 and m_2 is different, there exists a unique (δ, w_0) that $\mathbb{E}_{w_0, \delta \oplus w_0}(m_1) = s_1$ and $\mathbb{E}_{w_0, \delta \oplus w_0}(m_2) = s_2$.

Proof. The state obtained by excuting the Eagle encoding algorithm with $L-1$ bits on m_1 is $s_{L-1,1} = h_{L-1,1}(w_0) \oplus g_{L-1,1}(m_1, \delta)$, the state obtained by excuting the Eagle encoding algorithm with $L-1$ bits on m_2 is $s_{L-1,2} = h_{L-1,2}(w_0) \oplus g_{L-1,2}(m_2, \delta)$.

Due to the difference in parity between m_1 and m_2 , it can be assumed that the last bit of m_1 is 0 and the last bit of m_2 is 1. Further execute the Eagle encoding algorithm with one more bit separately to obtain

$$\begin{aligned} s_1 &= s_{L,1} = w_0^{+(L-1)} \oplus (g_{L-1,1}(m_1, \delta) \oplus g_{L-1,1}(m_1, \delta)^{+1}) \\ s_2 &= s_{L,2} = w_0^{+(L-1)} \oplus \delta \oplus (g_{L-1,2}(m_2, \delta) \oplus g_{L-1,2}(m_2, \delta)^{+1}) \end{aligned}$$

Perform XOR operation on both sides of the above two equations simultaneously, set $(g_{L-1,1}(m_1, \delta) \oplus g_{L-1,2}(m_2, \delta))$ as $g(m_1, m_2, \delta)$ to obtain

$$s_1 \oplus s_2 = \delta \oplus (g(m_1, m_2, \delta) \oplus g(m_1, m_2, \delta)^{+1})$$

Obviously, the left side of the equation is a definite number, on the right side of the equation is the result of performing XOR operation on an odd number of δ^{+k} , It is not difficult to verify that the solution to such an equation exists and is unique. $\square$

The conclusion of Theorem 4 indicates that, for any $\mathbb{E}_{w_0, w_1}(m_1) = s_1$ and $\mathbb{E}_{w_0, w_1}(m_2) = s_2$, when only given m_1, s_1, m_2, s_2 , there is a 50% probability that a unique w_0, w_1 valid.

[Theorem 5] For any two groups m_1, m_2 and m'_1, m'_2 , if $m_1 \oplus m_2 = m'_1 \oplus m'_2$, excute the Eagle encoding algorithm as (4.2) on m_1, m_2, m'_1, m'_2 to obtain

$$\begin{aligned}
\mathbb{E}_{w_0, \delta \oplus w_0}(m_1) &= w_0^{+(L-1)} \oplus g_L(m_1, \delta) \\
\mathbb{E}_{w_0, \delta \oplus w_0}(m_2) &= w_0^{+(L-1)} \oplus g_L(m_2, \delta) \\
\mathbb{E}_{w_0, \delta \oplus w_0}(m'_1) &= w_0^{+(L-1)} \oplus g_L(m'_1, \delta) \\
\mathbb{E}_{w_0, \delta \oplus w_0}(m'_2) &= w_0^{+(L-1)} \oplus g_L(m'_2, \delta)
\end{aligned} \tag{4.3}$$

then $g_L(m_1, \delta) \oplus g_L(m_2, \delta) = g_L(m'_1, \delta) \oplus g_L(m'_2, \delta)$ holds.

Proof. Returning to the calculation process of (4.2), it is not difficult to find that

$$\mathbb{E}_{w_0, \delta \oplus w_0}(m \oplus \Delta m) = w_0^{+(L-1)} \oplus g_L(m, \delta) \oplus j(\delta, \Delta m)$$

Where $j(\delta, \Delta m)$ is a function only related to $\delta, \Delta m$. Without loss of generality we set $m_1 \oplus m_2 = m'_1 \oplus m'_2 = \Delta m$, (4.2) can be reorganized as

$$\begin{aligned}
\mathbb{E}_{w_0, \delta \oplus w_0}(m'_1) &= w_0^{+(L-1)} \oplus g_L(m_1, \delta) \oplus j(\delta, \Delta m) \\
\mathbb{E}_{w_0, \delta \oplus w_0}(m'_2) &= w_0^{+(L-1)} \oplus g_L(m_2, \delta) \oplus j(\delta, \Delta m)
\end{aligned} \tag{4.4}$$

By simple calculate we can obtain

$$\begin{aligned}
&g_L(m'_1, \delta) \oplus g_L(m'_2, \delta) \\
&= g_L(m_1, \delta) \oplus g_L(m_2, \delta) \oplus (w_0^{+(L-1)} \oplus w_0^{+(L-1)}) \oplus (j(\delta, \Delta m) \oplus j(\delta, \Delta m)) \\
&= g_L(m_1, \delta) \oplus g_L(m_2, \delta)
\end{aligned} \quad \square$$

Theorem 5 indicates that, the function $g_L(m, \delta)$ satisfies the distributive law for both variables m and δ .

[Theorem 6] For the function $g_L(m, \delta)$ in (4.2), fix any δ , where δ contains an odd number of bits that are 1. Fix any u , there exists a unique m that $g_L(m, \delta) = u$.

Proof. Fix any δ , set $w_0 \in \{0, 1\}^L$, $w_1 = w_0 \oplus \delta$, excute Eagle encoding algorithm for m , we obtain $\mathbb{E}_{w_0, w_1}(m) = s$, then $u = s \oplus w_0^{+L}$, $g_L(m, \delta) = u$, thus $m \rightarrow u$ is monomorphism.

On the other hand, fix any δ , set $w_0 \in \{0, 1\}^L$, $w_1 = w_0 \oplus \delta$, excute Eagle decoding algorithm for $s = u \oplus w_0^{+L}$, we obtain $\mathbb{D}_{w_0, w_1}(s) = m$, thus $u \rightarrow m$ is monomorphism.

Furthermore, since the ranges of m and u are the same, the mapping from m to s is a one-to-one corresponding when δ is fixed.

□

The proof process of Theorem 6 actually describes the process of $g_L(m, \delta) = u$ and its inverse function $g_L^{-1}(u, \delta)$.

4.1 The multiplication operation of binary

According to the conclusion of Theorem 5, the symbol $g_L(m, \delta)$ satisfies the distributive law for both m and δ , that is

$$\begin{aligned} g_L(m, \delta_1 \oplus \delta_2) &= g_L(m, \delta_1) \oplus g_L(m, \delta_2) \\ g_L(m_1 \oplus m_2, \delta) &= g_L(m_1, \delta) \oplus g_L(m_2, \delta) \end{aligned}$$

If we consider the XOR operation $\oplus$ as an addition operation to binary, $g_L(m, \delta)$ can be seen as the multiplication operation between m and δ , as follows

$$g_L(m, \delta) = m * \delta$$

It is not difficult to verify that $m * \delta$ does not satisfy the commutative law or the associative law, so $m * \delta$ can only be seen as the right multiplication of δ by m . It is obvious that this representation is well-defined.

The calculation of $m * \delta$ is shown in Algorithm 3 as follows.

Algorithm 3 calculation of $m * \delta$.

Input:

Input: m, δ .

Output:

Output: $m * \delta$.

- 1: $w_0 \leftarrow 0$.
 - 2: $w_1 \leftarrow \delta$.
 - 3: $s_L \leftarrow \mathbb{E}_{w_0, w_1}(m)$.
 - 4: **return** s_L .
-

Similarly, we can also introduce the division operator $/$, using the notation $S/\delta = R$ to represent $R * \delta = S$. The calculation for division is as follows.

Algorithm 4 calculation of s/δ .

Input:

Input: s, δ .

Output:

Output: s/δ .

- 1: $w_0 \leftarrow 0$.
 - 2: $w_1 \leftarrow \delta$.
 - 3: $m \leftarrow \mathbb{D}_{w_0, w_1}(s)$.
 - 4: **return** m .
-

This paper does not intend to delve into the algebraic properties of the right multiplication operation mentioned above, but only simplifies the representation of $g_L(m, \delta)$ using $m * \delta$. Therefore $\mathbb{E}_{w_0, w_1}(m)$ can be expressed as $m * (w_0 \oplus w_1) \oplus w_0^{+(L-1)}$.

At the end of this section, we will provide the following conclusions.

[Theorem 7] If m is an odd number, then $m * \delta$ contains odd number of bits that are 1.

Proof. By definition, we have $m * \delta = \mathbb{E}_{0, \delta}(m)$. Let s_{L-1} denote the result of the penultimate step of performing Eagle encoding on m , so we have $s_L = \delta \oplus (s_{L-1} \oplus s_{L-1}^{+1})$.

Obviously $(s_{L-1} \oplus s_{L-1}^{+1})$ contains even number of bits that are 1, δ contains odd number of bits that are 1, thus s_L contains odd number of bits that are 1. $\square$

[Theorem 8] If m is an odd number, s contains odd number of bits that are 1, then there exists a unique δ that $m * \delta = s$.

Proof. According to theorem 4, set $m_1 = m, m_2 = 0, s_1 = s, s_2 = 0$, because the parity between m_1 and m_2 is different, there exists a unique $(0, \delta)$ that $\mathbb{E}_{0, \delta}(m) = s$ which can be equivalently expressed as $m * \delta = s$. $\square$

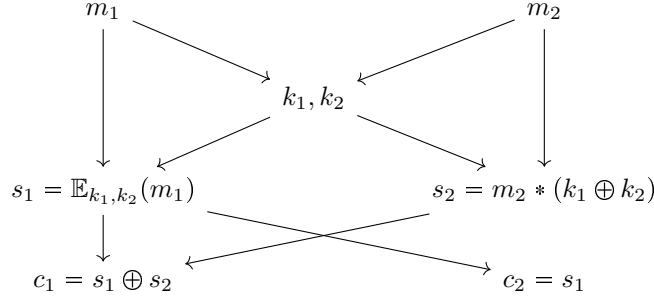
4.2 Introduction of a binary operation of two-dimensional vectors

We define a binary operation of two-dimensional vecotors $(m_1, m_2) \otimes (k_1, k_2) = (c_1, c_2)$ as follows.

$$\begin{aligned} c_2 &= \mathbb{E}_{k_1, k_2}(m_1) \\ c_1 &= c_2 \oplus (m_2 * (k_1 \oplus k_2)) \end{aligned} \tag{4.5}$$

Where k_1 contains even number of bits that are 1, k_2 contains odd number of bits that are 1, m_1, m_2 are both odd numbers, c_1 contains even number of bits that are 1, c_2 contains odd number of bits that are 1.

The above operation can be represented by the following figure.



It is not difficult to verify that the binary operation defined above satisfies the following conditions.

- (a) Given any (m_1, m_2) , the mapping $(k_1, k_2) \rightarrow (c_1, c_2)$ is a bijection.
- (b) Given any (k_1, k_2) , the mapping $(m_1, m_2) \rightarrow (c_1, c_2)$ is a bijection.
- (c) Given any (c_1, c_2) , the mapping $(k_1, k_2) \rightarrow (m_1, m_2)$ is a bijection.
- (d) Given only m_1 and c_1 , any (k_1, k_2) can be used to solve for a unique m_2 and c_2 .
- (e) Given only m_2 and c_2 , any (k_1, k_2) can be used to solve for a unique m_1 and c_1 .
- (f) Given only m_2 and c_1 , any (k_1, k_2) can be used to solve for a unique m_1 and c_2 .
- (g) Given only m_1 and c_2 , any k_1 can be used to solve for a unique k_2 , any k_2 can also be used to solve for a unique k_1 , any $k_1 \oplus k_2$ can also be used to solve for a unique k_1 or k_2 , and it is impossible to solve for m_2 or c_1 given any k_1 or k_2 or $k_1 \oplus k_2$.
- (h) Given only m_1, c_1, c_2 , any k_1 can be used to solve for a unique k_2, m_2 , any k_2 can also be used to solve for a unique k_1, m_2 , any $k_1 \oplus k_2$ can also be used to solve for a unique k_1 or k_2 .
- (i) Given only c_1, k_1, k_2 , any m_2 can be used to solve for a unique c_2 , any c_2 can also be used to solve for a unique m_2 .

For condition (a), It is obvious that $(k_1, k_2) \rightarrow (c_1, c_2)$ is an injective, but it is not so obvious that $(c_1, c_2) \rightarrow (k_1, k_2)$ be also an injective. Since $c_1 \oplus c_2 = m_2 * (k_1 \oplus k_2)$, from Theorem 8, we can solve an unique $k_1 \oplus k_2$. By $c_2 = \mathbb{E}_{k_1, k_2}(m_1) = m_1 * (k_1 \oplus k_2) \oplus k_0^{+(L-1)}$, we can solve an unique k_1 , thus the unique k_2 can be solved.

For other conditions (b)-(i), we can verify them in a similar way, we won't go into details here.

If we construct an encryption algorithm where (k_1, k_2) is the key, m_1 is the plaintext, c_1 is the ciphertext, m_2 and c_2 never appear in the plaintext sequences or the ciphertext sequences, according to the condition (d), for a cracker, under the condition of only knowing the plaintext ciphertext correspondence m_1, c_1 , any value (k_1, k_2) in the key space is valid and is indistinguishable, this feature allows us to construct the security that meets the requirements of computational complexity.

We summarize it as the following theorem.

[**Theorem 9**] As (4.5) defined, for any equation $(m_1, *) \otimes (k_1, k_2) = (c_1, *)$, which (k_1, k_2) are unknown variables and m_1, c_1 are known variables, this equation can be seen as an identity, valid for all k_1, k_2 .

Proof. This theorem requires us to construct an algorithm that, given any (k_1, k_2) to find a valid equation $(m_1, *) \otimes (k_1, k_2) = (c_1, *)$.

When given any (k_1, k_2) , we can solve c_2 and m_2 through the following method.

$$\begin{aligned} c_2 &= \mathbb{E}_{k_1, k_2}(m_1) \\ m_2 &= (c_1 \oplus c_2) / (k_1 \oplus k_2) \end{aligned}$$

Additionally, due to known m_1, c_1 , each m_2, c_2 can solve an unique $k_1, k_2, *$ in $(m_1, *) \otimes (k_1, k_2) = (c_1, *)$ can take all values without distinction, thus the equation $(m_1, *) \otimes (k_1, k_2) = (c_1, *)$ can be seen as an identity equation to unknown variables k_1 and k_2 .

□

5 Construction of one-way functions

5.1 Introduction to one-way functions

Before constructing the one-way function, we briefly introduce the properties of one-way function and the relationships with the P vs NP problem.

[**Definition 1**] A function is a one-way function means that the function satisfies the following properties:

- a) For a given x , there exists a polynomial-time algorithm that output $f(x)$.
- b) Given y , it is difficult to find an x that satisfies $y = f(x)$, that is, there does not exists a polynomial-time algorithm that finding the x .

The NP-complete problem refers to a set of problems that are verifiable in polynomial-time algorithm. For all NP-complete problems, whether there exists algorithms that are solvable in polynomial-time, this is the P vs NP problem. If $P \neq NP$, then for some NP problems, there is no algorithm that is solvable in polynomial-time. If $P = NP$, then for all NP problems, there exists algorithms that are solvable in polynomial-time.

If one-way function exists, it means that there exists such an NP problem, which has no deterministic polynomial time solvable algorithm, that is, $P \neq NP$. This is a direct inference, which can be directly described as the following theorem, See [9] for details.

[**Theorem 10**] If one-way function exists, then $P \neq NP$.

5.2 Construction of one-way functions

For short key encryption algorithms, the security of the key depends on the computational complexity of cracking the key when given the known plaintext-ciphertext pairs. In theory, if the key can only be cracked through exhaustive search, this algorithm is considered computationally secure. However, currently all short key encryption algorithms, with known plaintext-ciphertext pairs, have no evidence to suggest that attackers can crack the key through only exhaustive search.

In this chapter, we constructed a short key symmetric encryption algorithm, which we named it Eagle. The key consists of six independent variables, each of which contains at least 2^{L-1} possible values. During the encryption process, several intermediate random numbers are generated at the encryption party, after the encryption process is completed, these intermediate random numbers are directly deleted at the encryption party, and only the ciphertext is sent to the decryption party. Under the condition of knowing the correspondence between any multiple sets of plaintext-ciphertext, the problem of cracking the key is equivalent to solving a system of equations. In the process of solving this system of equations, at least one unknown variables cannot be eliminated, so it is necessary to traverse all possible values of at least this variable, which means that the computational complexity for cracking the key is at least $O(2^{L-1})$.

On the other hand, when the key is known, both the encryption and decryption processes can be completed in linear time to the length of the key, meaning that the computational complexity for both encryption and decryption is $O(KL)$, where K is a constant.

Furthermore, since the decryption process under the condition of knowing the ciphertext and key is reversible for the problem of cracking the key under the condition of knowing the correspondence between plaintext and ciphertext, the decryption process can be completed in $O(KL)$, while the cracking process is completed at least in $O(2^L)$, thus satisfying the condition of one-way function.

It should be noted that although the computational complexity of cracking the key is exponentially under the condition of knowing the correspondence between plaintext and ciphertext, this does not mean that this encryption algorithm is safe and practical. Assume the length of the key is $6L - 6$ bits, although the computational complexity of cracking the key is not less than $O(2^{L-1})$, may be much lower than the expected complexity of $O(2^{6L-6})$. A secure and practical encryption algorithm needs to ensure that the cracking key can only exhaustively search for every possible key which is the issue to be considered in the next chapter. In this chapter, our goal is to design a one-way function where we only need to design and prove that the computational complexity for cracking keys is exponential to the length of the key.

Next, we will describe it in five sections. Section 5.2.1 introduces the key

generation of the Eagle encryption algorithm, Section 5.2.2 introduces the process of the Eagle encryption algorithm, Section 5.2.3 introduces the process of the Eagle decryption algorithm, Section 5.2.4 analyzes the correctness and complexity of the Eagle encryption algorithm, and Section 5.2.5 provides the security analysis of the Eagle algorithm which give a construction of one-way function.

5.2.1 Eagle key generator

The key is (w_0, w_1) , (w_2, w_3) and (w_4, w_5) , where w_0, w_2, w_4 both contains even number of bits that are 1, w_1, w_3, w_5 both contains odd number of bits that are 1. That is to say w_0 and w_1 have odd number of bits different, w_2 and w_3 have odd number of bits different, w_4 and w_5 have odd number of bits different.

Assuming $w_0, w_1, w_2, w_3, w_4, w_5$ are all L bits, the length of the entire key can be considered as $6L - 6$ bits, this is because they have a loss of 6 bit.

5.2.2 Eagle encryption process

The plaintext P is grouped by L bits, and the last group with less than L bits are randomly filled into L bits. The total number of groups is assumed to be K , denoted as $P = (P_1, P_2, \dots, P_K)$.

Then generate random numbers of $6K + 2$ groups with each have L bits, denoted as $(z_1, z_2, \dots, z_K)$, $(t_1, t_2, \dots, t_K)$, $(x_1, x_2, \dots, x_{2K}, x_{2K+1})$, $(y_1, y_2, \dots, y_{2K}, y_{2K+1})$. where $x_i (1 \leq i \leq 2K + 1)$ contains even number of bits that are 1, $y_i (1 \leq i \leq 2K + 1)$ contains odd number of bits that are 1, $t_i (1 \leq i \leq K)$ are odd numbers, $z_i (1 \leq i \leq K)$ are odd numbers.

Generate another four random numbers with each have L bits denoted as M_0, P_{K+1}, u and v , where M_0 is an odd number, P_{K+1} is an odd number, u contains even number of bits different, v contains odd number of bits that are 1.

Next, we need to define two simple functions as follows.

$$\phi(x) = \begin{cases} x, & \text{if } x \text{ is an odd number} \\ x \oplus 1, & \text{if } x \text{ is an even number} \end{cases} \quad (5.1)$$

$$\psi(x) = \begin{cases} x, & \text{if } x \text{ contains odd number of bits that are 1} \\ x \oplus 1, & \text{if } x \text{ contains even number of bits that are 1} \end{cases} \quad (5.2)$$

Where $\phi(x)$ changes x to an odd number by modifying the last bit of x , $\psi(x)$ changes x to a number containing odd number of bits that are 1 by modifying the last bit of x .

The encryption process is described in the following algorithm.

Algorithm 5 *Eagle-Encrypt*.

Input:

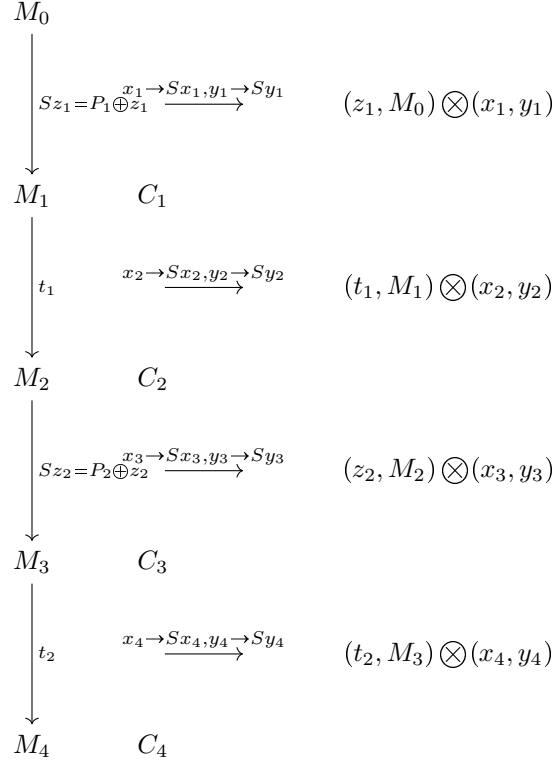
Key: $(w_0, w_1), (w_2, w_3), (w_4, w_5)$.
Plaintext sequence: $P = (P_1, P_2, \dots, P_K)$.
Random number sequence:
 $(z_1, z_2, \dots, z_K)$,
 $(t_1, t_2, \dots, t_K)$,
 $(x_1, x_2, \dots, x_{2K+1})$,
 $(y_1, y_2, \dots, y_{2K+1})$.
Random number: M_0, P_{K+1}, u, v .

Output:

Ciphertext sequence:
 $(C_1, C_2, \dots, C_{2K+1})$,
 $(Sx_1, Sx_2, \dots, Sx_{2K+1})$,
 $(Sy_1, Sy_2, \dots, Sy_{2K+1})$,
 $(Sz_1, Sz_2, \dots, Sz_K)$,
 Su, Sv, P_{K+1} .

```
1:  $Su \leftarrow \mathbb{E}_{w_4, w_5}(u)$ 
2:  $Sv \leftarrow \mathbb{E}_{w_4, w_5}(v)$ 
3:  $M \leftarrow M_0$ 
4:  $C \leftarrow 0$ 
5: for  $i = 1; i \leq K; i++$  do
6:    $Sz_i \leftarrow P_i \oplus z_i$ 
7:    $(C_{2i-1}, C) \leftarrow (z_i, M) \otimes (x_{2i-1}, y_{2i-1})$ 
8:    $M \leftarrow \mathbb{E}_{u, v}(C)$ 
9:    $M \leftarrow \phi(M)$ 
10:   $(C_{2i}, C) \leftarrow (t_i, M) \otimes (x_{2i}, y_{2i})$ 
11:   $M \leftarrow \mathbb{E}_{u, v}(C)$ 
12:   $M \leftarrow \phi(M)$ 
13: end for
14:  $(C_{2K+1}, C) \leftarrow (P_{K+1}, M) \otimes (x_{2K+1}, y_{2K+1})$ 
15: for  $j = 1; j \leq 2K+1; j++$  do
16:    $Sx_j \leftarrow \mathbb{E}_{w_0, w_1}(x_j)$ 
17:    $Sy_j \leftarrow \mathbb{E}_{w_2, w_3}(y_j)$ 
18: end for
```

The above algorithm can be represented by the following figure.



5.2.3 Eagle decryption process

For the decrypting party, the key is also $(w_0, w_1), (w_2, w_3), (w_4, w_5)$, the ciphertext is $(C_1, C_2, \dots, C_{2K+1}), (Sx_1, Sx_2, \dots, Sx_{2K+1}), (Sy_1, Sy_2, \dots, Sy_{2K+1}), (Sz_1, Sz_2, \dots, Sz_K), Su, Sv, P_{K+1}$. Please refer to the following for details.

Algorithm 6 *Eagle*-Decrypt.

Input:Key: $(w_0, w_1), (w_2, w_3), (w_4, w_5)$.

Ciphertext sequence:

 $(C_1, C_2, \dots, C_{2K+1}),$
 $(Sx_1, Sx_2, \dots, Sx_{2K+1}),$
 $(Sy_1, Sy_2, \dots, Sy_{2K+1}),$
 $(Sz_1, Sz_2, \dots, Sz_K),$
 $Su, Sv, P_{K+1}.$ **Output:**Plaintext sequence: $P = (P_1, P_2, \dots, P_K).$

```
1:  $u \leftarrow \mathbb{D}_{w_4, w_5}(Su)$ 
2:  $v \leftarrow \mathbb{D}_{w_4, w_5}(Sv)$ 
3: for  $j = 1; j \leq 2K + 1; j++$  do
4:    $x_j \leftarrow \mathbb{D}_{w_0, w_1}(Sx_j)$ 
5:    $y_j \leftarrow \mathbb{D}_{w_2, w_3}(Sy_j)$ 
6: end for
7:  $C \leftarrow \mathbb{E}_{x_{2K+1}, y_{2K+1}}(P_{K+1})$ 
8:  $M \leftarrow (C \oplus C_{2K+1}) / (x_{2K+1} \oplus y_{2K+1})$ 
9:  $C \leftarrow \mathbb{D}_{u, v}(M)$ 
10:  $C \leftarrow \psi(C)$ 
11: for  $i = K; i \geq 1; i--$  do
12:    $M \leftarrow (C \oplus C_{2i}) / (x_{2i} \oplus y_{2i})$ 
13:    $C \leftarrow \mathbb{D}_{u, v}(M)$ 
14:    $C \leftarrow \psi(C)$ 
15:    $z_i \leftarrow \mathbb{D}_{x_{2i-1}, y_{2i-1}}(C)$ 
16:    $P_i \leftarrow z_i \oplus Sz_i$ 
17:    $M \leftarrow (C \oplus C_{2i-1}) / (x_{2i-1} \oplus y_{2i-1})$ 
18:    $C \leftarrow \mathbb{D}_{u, v}(M)$ 
19:    $C \leftarrow \psi(C)$ 
20: end for
```

5.2.4 Basic analysis of *Eagle*

It is obvious that *Eagle* encryption algorithm is correct because the decryption process and encryption process are mutually inverse.

From the encryption process above, it can be seen that the *Eagle* encryption algorithm is divided into two stages: the preparation stage and the encryption stage. In the preparation stage, different parameters are randomly generated. In the encryption stage, the ciphertexts generated by the random parameters and the plaintexts together with the keys. After the encryption process is completed, these independent random parameters are directly deleted by the encryption party and do not appear in the ciphertexts.

It should also be noted that after being encrypted by the *Eagle* encryption

algorithm, the length of the ciphertext is about 7 times the length of the plaintext. This is because in the preparation stage of the *Eagle* encryption algorithm, independent random numbers with a length of about 6 times the plaintext are generated, and then these random numbers are encrypted using the known short key. The encrypted ciphertext formed by these random numbers is also bound to the final ciphertexts.

The *Eagle* encryption algorithm performs bit by bit in encryption and decryption process, and the computational complexity of operation for each bit is also a constant level. Therefore, the encryption and decryption complexity of the *Eagle* encryption algorithm can be regarded as $O(KL)$.

5.2.5 A construction of one-way function

When given the ciphertext $(C_1, C_2, \dots, C_{2K+1})$, $(Sx_1, Sx_2, \dots, Sx_{2K+1})$, $(Sy_1, Sy_2, \dots, Sy_{2K+1})$, $(Sz_1, Sz_2, \dots, Sz_K)$, and (Su, Sv, P_{K+1}) of the *Eagle* encryption algorithm, we define the following function.

$$f(w_0, w_1, w_2, w_3, w_4, w_5) = (P_1, P_2, \dots, P_K) \quad (5.3)$$

where $(w_0, w_1, w_2, w_3, w_4, w_5)$ is the short key and $(P_1, P_2, \dots, P_K)$ is the plaintext decrypted by the *Eagle* decryption algorithm.

The inverse function of f can be defined as

$$\begin{aligned} f^{-1}(P_1, P_2, \dots, P_K) &= \{(w_0, w_1, w_2, w_3, w_4, w_5) | \\ &f(w_0, w_1, w_2, w_3, w_4, w_5) = (P_1, P_2, \dots, P_K)\} \end{aligned} \quad (5.4)$$

Solving f^{-1} is equivalent to finding the key $(w_0, w_1, w_2, w_3, w_4, w_5)$ to match the plaintext $(P_1, P_2, \dots, P_K)$.

In fact, if we can prove that we can't solve f^{-1} in polynomial time, that is, for any attacker who only knows the plaintext-ciphertext pairs, there is no effective method in polynomial time to find the correct key.

Here we need to clarify that we only need to prove that we can't solve $w = (w_0, w_1, w_2, w_3, w_4, w_5)$ in polynomial time, instead of relying only exhaustive search to find w . In fact, "only exhaustive search can be used to find w " is much stricter semantically than "unable to solve w in polynomial time". In this chapter, we only need to prove that w can't be solved in polynomial time, and this already satisfies the condition of one-way function.

Intuitively, we can't solve (w_0, w_1) and $(x_1, x_2, \dots, x_{2K+1})$ when only $(Sx_1, Sx_2, \dots, Sx_{2K+1})$ are given, we can't solve (w_2, w_3) and $(y_1, y_2, \dots, y_{2K+1})$ when only $(Sy_1, Sy_2, \dots, Sy_{2K+1})$ are given, we can't solve (w_4, w_5) when only Su, Sv are given, when given P_i, Sz_i, C_i , we can obtain the equation $(P_i \oplus Sz_i, *) \otimes (x_i, y_i) = (C_i, *)$, according to the theorem 9, this is an identity equation.

Based on the above analysis, given the correspondence between the plaintext $(P_1, P_2, \dots, P_K)$ and ciphertext $(C_1, C_2, \dots, C_{2K+1})$, $(Sx_1, Sx_2, \dots, Sx_{2K+1})$,

$(Sy_1, Sy_2, \dots, Sy_{2K+1}), (Sz_1, Sz_2, \dots, Sz_K), Su, Sv, P_{K+1}$ of the Eagle encryption algorithm, the problem of solving the key $(w_0, w_1, w_2, w_3, w_4, w_5)$ is equivalent to the problem of solving the respective keys in three independent encryption algorithms under the condition that only their ciphertext is known. The first encryption algorithm solves its key (w_4, w_5) under the condition that its ciphertext (Su, Sv) are known. For the second encryption algorithm, the key (w_0, w_1) need to be solved under the condition that only the ciphertext $(Sx_1, Sx_2, \dots, Sx_{2K+1})$ is known. For the third encryption algorithm, the key (w_2, w_3) need to be solved under the condition that only the ciphertext $(Sy_1, Sy_2, \dots, Sy_{2K+1})$ is known.

In fact, for these three independent encryption algorithms, under the condition of only knowing their respective ciphertext, every possible key cannot be reasonably excluded, and every possible key must be exhaustively searched. From the perspective of the entire Eagle encryption algorithm, at least one set of keys from (w_0, w_1) or (w_2, w_3) or (w_4, w_5) needs to be exhaustively searched to find a valid key. The computational complexity of exhausting either (w_0, w_1) or (w_2, w_3) or (w_4, w_5) is exponentially.

Next, let's roughly introduce the idea of the proof. During the encryption process, all generated intermediate states are unknown in the ciphertext, and these intermediate states need to be gradually restored during the decryption process.

Without specifying any intermediate variables such as M or C , the problem of solving f^{-1} is equivalently transformed into solving a system of equations about the unknown variables as $(w_0, w_1, w_2, w_3, w_4, w_5), (x_1, x_2, \dots, x_{2K+1}), (y_1, y_2, \dots, y_{2K+1})$, any equation in this system of equations can be regarded as an identity equation about the unknown variables, and it is impossible to find an effective solution.

On the other hand, given $(P_1, P_2, \dots, P_K)$ and $(C_1, C_2, \dots, C_{2K+1}), (Sx_1, Sx_2, \dots, Sx_{2K+1}), (Sy_1, Sy_2, \dots, Sy_{2K+1}), (Sz_1, Sz_2, \dots, Sz_K), Su, Sv, P_{K+1}$, every intermediate state M or C can take any value, the computational complexity of traversing every intermediate state is $O(2^{L-1})$.

This indicates that the computational complexity of solving f^{-1} is at least $O(2^{L-1})$.

Let's give the following theorem.

[Theorem 11] When given the ciphertext $(C_1, C_2, \dots, C_{2K+1}), (Sx_1, Sx_2, \dots, Sx_{2K+1}), (Sy_1, Sy_2, \dots, Sy_{2K+1}), (Sz_1, Sz_2, \dots, Sz_K), Su, Sv, P_{K+1}$ and the plaintext $(P_1, P_2, \dots, P_K)$ of the *Eagle* encryption algorithm, the computational complexity of the problem of cracking the key $(w_0, w_1, w_2, w_3, w_4, w_5)$ is at least $O(2^{L-1})$.

Proof. Before the formal proof, we first introduce a simple formal language for the process of solving equations, we use $\langle x, y \rangle_e$ to describe the relationship between the two unknown variables x and y during the process of solving equation e . If the equation e is an identify equation for x, y , we denote it as

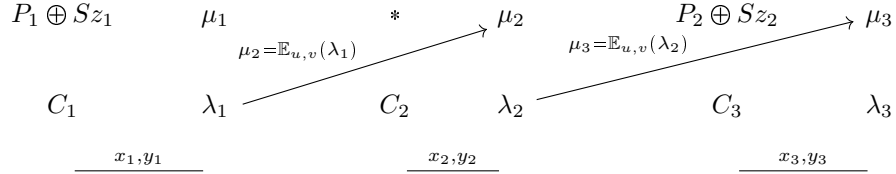
$\langle x, y \rangle_e = 1$. If the equation e can be solved for x, y or it can be determined that there is no solution, we denote it as $\langle x, y \rangle_e = 0$. If the equation e can be partially solved for x, y , meaning that from equation e , for any given x , the variable y can be solved, or for any given y , the variable x can be solved, we denote it as $\langle x, y \rangle_e = -1$. The formal definition of $\langle x, y \rangle_e$ is as follows.

$$\langle x, y \rangle_e = \begin{cases} 1, & \text{if } e \text{ is an identity equation} \\ 0, & \text{if } x, y \text{ can be solved or there is no solution} \\ -1, & \text{if } x, y \text{ can be partially solved} \end{cases} \quad (5.5)$$

If we formulate two equations e_1, e_2 simultaneously, we can regard $\langle x, y \rangle_{e_1, e_2}$ as a multiplication operation of $\langle x, y \rangle_{e_1}$ and $\langle x, y \rangle_{e_2}$, which denote as $\langle x, y \rangle_{e_1, e_2} = \langle x, y \rangle_{e_1} \times \langle x, y \rangle_{e_2}$. Obvious, we have $1 \times \gamma = \gamma$, $0 \times \gamma = 0$. It should be noted that -1×-1 is undefined, it may be equal to 0 or -1, in most cases, it can be assumed that $-1 \times -1 = 0$, this is because each of the two equations can solve for some values of x, y , when combined, they can solve for most cases of x, y .

In this paper, we only introduce this formal language for solving equations to facilitate the description of the proof process, without delving into the analysis of the formal language itself.

By observing the structure of Algorithm 5 and Algorithm 6, when i is an odd number, the plaintext ciphertext pair of the i -th group satisfy the equation $(P_i \oplus Sz_i, \mu_i) \otimes (x_i, y_i) = (C_i, \lambda_i)$, where μ_i, λ_i are used to represent the intermediate variables as M or C . When i is an even number, the plaintext ciphertext pair of the i -th group satisfy the equation $(*, \mu_i) \otimes (x_i, y_i) = (C_i, \lambda_i)$. The structure of the entire parameter can be represented as shown in the figure below.



Where $x_i = \mathbb{D}_{w_0, w_1}(Sx_i)$ and $y_i = \mathbb{D}_{w_2, w_3}(Sy_i)$. From the above, the problem of cracking the key $(w_0, w_1, w_2, w_3, w_4, w_5)$ is equivalent to solving the following system of equations.

$$\left\{ \begin{array}{l} (P_1 \oplus Sz_1, \mu_1) \otimes (\mathbb{D}_{w_0, w_1}(Sx_1), \mathbb{D}_{w_2, w_3}(Sy_1)) = (C_1, \lambda_1) \\ (*, \mu_2) \otimes (\mathbb{D}_{w_0, w_1}(Sx_2), \mathbb{D}_{w_2, w_3}(Sy_2)) = (C_2, \lambda_2) \\ (P_2 \oplus Sz_2, \mu_3) \otimes (\mathbb{D}_{w_0, w_1}(Sx_3), \mathbb{D}_{w_2, w_3}(Sy_3)) = (C_3, \lambda_3) \\ (*, \mu_4) \otimes (\mathbb{D}_{w_0, w_1}(Sx_4), \mathbb{D}_{w_2, w_3}(Sy_4)) = (C_4, \lambda_4) \\ \dots\dots \\ (P_K \oplus Sz_K, \mu_{2K-1}) \otimes (\mathbb{D}_{w_0, w_1}(Sx_{2K-1}), \mathbb{D}_{w_2, w_3}(Sy_{2K-1})) = (C_{2K-1}, \lambda_{2K-1}) \\ (*, \mu_{2K}) \otimes (\mathbb{D}_{w_0, w_1}(Sx_{2K}), \mathbb{D}_{w_2, w_3}(Sy_{2K})) = (C_{2K}, \lambda_{2K}) \\ (P_{K+1}, \mu_{2K+1}) \otimes (\mathbb{D}_{w_0, w_1}(Sx_{2K+1}), \mathbb{D}_{w_2, w_3}(Sy_{2K+1})) = (C_{2K+1}, \lambda_{2K+1}) \end{array} \right. \quad (5.6)$$

$$\left\{ \begin{array}{l} \mathbb{E}_{u,v}(\lambda_1) = \mu_2 \\ \mathbb{E}_{u,v}(\lambda_2) = \mu_3 \\ \dots\dots \\ \mathbb{E}_{u,v}(\lambda_{2K-2}) = \mu_{2K-1} \\ \mathbb{E}_{u,v}(\lambda_{2K-1}) = \mu_{2K} \\ \mathbb{E}_{u,v}(\lambda_{2K}) = \mu_{2K+1} \end{array} \right. \quad (5.7)$$

$$\left\{ \begin{array}{l} \mathbb{E}_{w_4, w_5}(u) = Su \\ \mathbb{E}_{w_4, w_5}(v) = Sv \end{array} \right. \quad (5.8)$$

Where $(w_0, w_1, w_2, w_3, w_4, w_5)$, $(\lambda_1, \lambda_2, \dots, \lambda_{2K+1})$, $(\mu_1, \mu_2, \dots, \mu_{2K+1})$ and (u, v) are unknown variables to be solved, $(P_1, P_2, \dots, P_K)$, $(C_1, C_2, \dots, C_{2K+1})$, $(Sx_1, Sx_2, \dots, Sx_{2K+1})$, $(Sy_1, Sy_2, \dots, Sy_{2K+1})$, $(Sz_1, Sz_2, \dots, Sz_K)$, Su, Sv, P_{K+1} are known variables.

According to the theorem 4, if we can solve (u, v) from (5.6) and (5.7), we can solve an unique (w_4, w_5) from (5.8). Since (w_4, w_5) does not appear in (5.6) and (5.7), we can skip (5.8) in the later analysis.

Now we focus on the two unknown variables (u, v) for equations (5.6) and (5.7).

From (5.7), variables u, v has no constraints, thus $\langle u, v \rangle_{(5.7)} = 1$. On the other hand, the implicit relationship between the variables u, v and λ_i, μ_i shown in (5.7) is equivalent to the following expressions which requires determination by (5.6).

$$\langle u, v \rangle_{(5.6)} = \begin{cases} 1 & \iff \text{for all } i, \langle \lambda_i, \mu_{i+1} \rangle_{(5.6)} \neq 0 \\ 0 & \iff \text{at least two groups } \langle \lambda_i, \mu_{i+1} \rangle_{(5.6)} = 0 \\ -1 & \iff \text{only one group } \langle \lambda_i, \mu_{i+1} \rangle_{(5.6)} = 0 \end{cases} \quad (5.9)$$

If i is an even number, according to the condition (i) by the $\otimes$ operation descibed in chapter 4.2, from the i -th equation from (5.6), if (w_0, w_1, w_2, w_3) solved, we have $\langle \lambda_i, \mu_i \rangle_{(5.6)} = -1$, if (w_0, w_1, w_2, w_3) can't be solved, we have $\langle \lambda_i, \mu_i \rangle_{(5.6)} = 1$. Thus we can obtain the following equivalent conclusion.

$$\langle \lambda_i, \mu_i \rangle_{(5.6)} \neq 0 \iff i \text{ is an even number} \quad (5.10)$$

Since for all i , either i or $i+1$ must be an even number, thus $\langle \lambda_i, \mu_{i+1} \rangle_{(5.6)} \neq 0$ holds for all i . By (5.9), we can conclude that $\langle u, v \rangle_{(5.6)} \neq 0$. Since $\langle u, v \rangle_{(5.7)} = 1$, thus $\langle u, v \rangle_{(5.6, 5.7)} = 1$ or $\langle u, v \rangle_{(5.6, 5.7)} = -1$.

Based on the above analysis, at least one of the two variables (u, v) can't be solved from the equations, thus we need to exhaustive search one variable in (u, v) .

Since u or v can take at least 2^{L-1} values, we can conclude that the computational complexity of the problem of cracking the key $(w_0, w_1, w_2, w_3, w_4, w_5)$ is at least $O(2^{L-1})$. □

In the proof of Theorem 11, the fact that at least one variable cannot be directly solved from the system of equations does not imply that it is a free variable. In fact, when $K > 6$, the number of constraints in the system of equations (5.6), (5.7) exceeds the number of unknown variables, which means that the system of equations typically has only one solution.

According to the conclusion of Theorem 11, the computational complexity of solving f^{-1} is exponential.

Due to the fact that solving f can be completed in polynomial time and the computational complexity of solving f^{-1} is exponential, f is a one-way function, we can summarize it to the following theorem.

[Theorem 12] The function f defined in (5.3) is a one-way function.

According to Theorem 10 and Theorem 12, we conclude that $P \neq NP$.

6 How to achieve only exhaustive search to crack keys

Observing the equation $R * \delta * \delta = S$, where R and S are known variables, δ is the unknown variable to be solved. Since the left side of the equation performs two right multiplications on δ , this equation can be regarded as a “quadratic equation in one variable” about δ . Obviously, this is a nonlinear equation, and we have sufficient reason to make the following assertion.

[Assertion 1] The equation $R * \delta * \delta = S$ about δ has no quick solution.

Without causing ambiguity, we denote the operation of performing two right multiplications by δ on R as $R * \delta^2$.

Similarly, we can use S/δ^2 to represent performing two division operations on S with respect to δ .

Now we give the two algorithms $m * \delta^2$ and s/δ^2 as follows

Algorithm 7 calculation of $m * \delta^2$.

Input:Input: m, δ .**Output:**Output: $m * \delta^2$.1: $s \leftarrow m * \delta$ 2: $s \leftarrow \phi(s)$ 3: $s \leftarrow s * \delta$ 4: **return** s .

Algorithm 8 calculation of s/δ^2 .

Input:Input: s, δ .**Output:**Output: s/δ^2 .1: $m \leftarrow s/\delta$.2: $m \leftarrow \psi(m)$.3: $m \leftarrow m/\delta$.4: **return** m .

6.1 Design of encryption algorithm for only exhaustive method to crack the key

Although we found an encryption algorithm in Chapter 5 that, given the known correspondence between plaintext and ciphertext, the computational complexity for cracking the key is at least exponentially to the length of key as $O(2^{L-1})$, this does not mean that this algorithm is practical. Because the length of the key $(w_0, w_1, w_2, w_3, w_4, w_5)$ is $6L - 6$ bits, theoretically, the computational complexity for cracking the key must reach $O(2^{6L-6})$ to recognize its secure.

In this chapter, we utilized the Eagle encoding algorithm and its related features to design another short key symmetric block encryption algorithm, which we named the *Eagle** encryption algorithm. The main feature of this algorithm is that it introduce a nonlinear operation, together with the random numbers as designed in Eagle encryption algorithm.

After being encrypted by this encryption algorithm, the problem of cracking the keys given that all plaintext ciphertext correspondences are known, is equivalent to the problem of two independent encryption systems cracking their respective keys with only their ciphertext known.

In addition, for exhaustively searching for one of the two keys, solving the other key requires solving a nonlinear equation, which can only be solved by exhaustive search.

The entire *Eagle** encryption algorithm is divided into three parts: key generation, encryption algorithm, and decryption algorithm. Next, we will provide a detailed description.

6.1.1 *Eagle** key generator

The key is divided into two independent variables, denoted as δ_1, δ_2 , where δ_1, δ_2 both contain odd number of bits that are 1.

Assuming δ_1, δ_2 are all L bits, the length of the entire key can be considered as $2L - 2$ bits, this is because each variable has a loss of 1 bit.

6.1.2 *Eagle** encryption algorithm

The plaintext P is grouped by L bits, and the last group with less than L bits are randomly filled into L bits. The total number of groups is assumed to be K , denoted as $P = (P_1, P_2, \dots, P_K)$.

We randomly generate $3K + 2$ groups number with each have L bits, denoted as $x = (x_1, x_2, \dots, x_{K+1})$, $y = (y_1, y_2, \dots, y_{K+1})$, $z = (z_1, z_2, \dots, z_K)$, where x_i contains even number of bits that are 1, y_i contains odd number of bits that are 1, z_i is odd number.

Then we randomly generate four number as u, v, M_0 and P_{K+1} , where u contains even number of bits that are 1, v contains odd number of bits that are 1, M_0 is an odd number, P_{K+1} is an odd number.

The encryption process is detailed in Algorithm 9 below.

Algorithm 9 *Eagle**-Encrypt.

Input:Keys: (δ_1, δ_2) .Plaintext sequences: $P = (P_1, P_2, \dots, P_K)$.

Random number sequences:

$$x = (x_1, x_2, \dots, x_{K+1}),$$

$$y = (y_1, y_2, \dots, y_{K+1}).$$

$$z = (z_1, y_2, \dots, z_K).$$

Random number: u, v, M_0, P_{K+1} .**Output:**

Ciphertext sequences:

$$(C_1, C_2, \dots, C_{K+1}),$$

$$(Sx_1, Sx_2, \dots, Sx_{K+1}),$$

$$(Sy_1, Sy_2, \dots, Sy_{K+1}),$$

$$(Sz_1, Sz_2, \dots, Sz_K),$$

$$Su, Sv, P_{K+1}$$

$$1: Su \leftarrow (u \oplus 1) * \delta_1 * \delta_1$$

$$2: Sv \leftarrow v * \delta_2 * \delta_2$$

$$3: \textbf{for } i = 1; i \leq K + 1; i++ \textbf{ do}$$

$$4: \quad Sx_i \leftarrow (x_i \oplus 1) * \delta_1 * \delta_1$$

$$5: \quad Sy_i \leftarrow y_i * \delta_2 * \delta_2$$

$$6: \textbf{end for}$$

$$7: M \leftarrow M_0$$

$$8: C \leftarrow 0$$

$$9: \textbf{for } j = 1; j \leq K; j++ \textbf{ do}$$

$$10: \quad Sz_j \leftarrow P_j \oplus z_j$$

$$11: \quad (C_j, C) \leftarrow (z_j, M) \otimes (x_j, y_j)$$

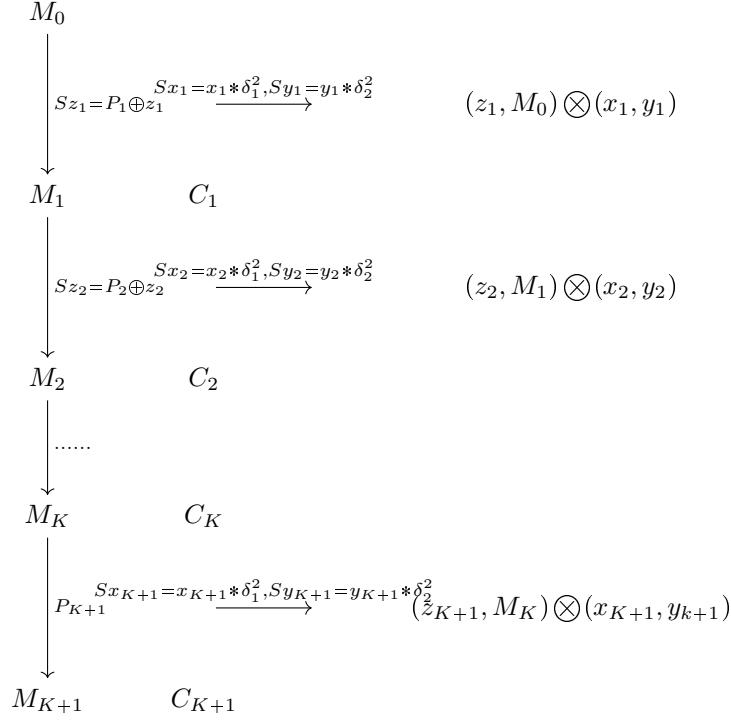
$$12: \quad M \leftarrow \mathbb{E}_{u,v}(C)$$

$$13: \quad M \leftarrow \phi(M)$$

$$14: \textbf{end for}$$

$$15: (C_{K+1}, C) \leftarrow (P_{K+1}, M) \otimes (x_{K+1}, y_{K+1})$$

From the above algorithm, it can be seen that the *Eagle** encryption algorithm is divided into two independent parts, as shown in the following figure.



As shown from the above figure, the ciphertext length is about three times the plaintext length.

6.1.3 *Eagle** decryption algorithm

For *Eagle** decryptors, the ciphertext sequences are $C = (C_1, C_2, \dots, C_{K+1}, Sx_1, Sx_2, \dots, Sx_{K+1}, Sy_1, Sy_2, \dots, Sy_{K+1}, Sz_1, Sz_2, \dots, Sz_K), Su, Sv$ and P_{K+1} . Here, the ciphertext sequences can be divided into three independent parts $(C_1, C_2, \dots, C_{K+1}, Sz_1, Sz_2, \dots, Sz_K, P_{K+1})$ and $(Sx_1, Sx_2, \dots, Sx_{K+1}, Su)$ and $(Sy_1, Sy_2, \dots, Sy_{K+1}, Sv)$. First we use (δ_1) and $(Sx_1, Sx_2, \dots, Sx_{K+1}, Su)$ to calculate the random number sequences $x = (x_1, x_2, \dots, x_{K+1}, u)$, then we use (δ_2) and $(Sy_1, Sy_2, \dots, Sy_{K+1}, Sv)$ to calculate the random number sequences $y = (y_1, y_2, \dots, y_{K+1}, v)$, then we use the random number u, v and the ciphertexts $(C_1, C_2, \dots, C_{K+1}, Sz_1, Sz_2, \dots, Sz_K, P_{K+1})$ and the random number sequences $x = (x_1, x_2, \dots, x_{K+1})$ and $y = (y_1, y_2, \dots, y_{K+1})$ to decrypt $P = (P_1, P_2, \dots, P_K)$. The specific process is detailed in the following algorithm.

Algorithm 10 *Eagle**-Decrypt.

Input:

Key: (δ_1, δ_2) .
Ciphertext sequences:
 $(C_1, C_2, \dots, C_{K+1})$,
 $(Sx_1, Sx_2, \dots, Sx_{K+1})$,
 $(Sy_1, Sy_2, \dots, Sy_{K+1})$,
 $(Sz_1, Sz_2, \dots, Sz_K)$,
 Su, Sv, P_{K+1}

Output:

Plaintext sequences: $P = (P_1, P_2, \dots, P_K)$.

```
1:  $M \leftarrow 0, C \leftarrow 0$ 
2:  $u \leftarrow (Su/\delta_1/\delta_1) \oplus 1$ 
3:  $v \leftarrow Sv/\delta_2/\delta_2$ 
4: for  $i = 1; i \leq K + 1; i++$  do
5:    $x_i \leftarrow (Sx_i/\delta_1/\delta_1) \oplus 1$ 
6:    $y_i \leftarrow Sy_i/\delta_2/\delta_2$ 
7: end for
8:  $C \leftarrow \mathbb{E}_{x_{K+1}, y_{K+1}}(P_{K+1})$ 
9:  $M \leftarrow (C_{K+1} \oplus C)/(x_{K+1} \oplus y_{K+1})$ 
10: for  $j = K; j \geq 1; j--$  do
11:    $C \leftarrow \mathbb{D}_{u,v}(M)$ 
12:    $C \leftarrow \psi(C)$ 
13:    $z_j \leftarrow \mathbb{D}_{x_j, y_j}(C)$ 
14:    $M \leftarrow (C_j \oplus C)/(x_j \oplus y_j)$ 
15:    $P_j \leftarrow z_j \oplus Sz_j$ 
16: end for
```

From the processes of Algorithm 9 and Algorithm 10, it can be seen that the *Eagle** encryption and *Eagle** decryption are mutually inverse, which means that as an encryption algorithm, *Eagle** is correct.

In addition, it is easy to see that the calculations are all completed in linear time, so the computational complexity of Algorithm 9 and Algorithm 10 are all $O(KL)$.

6.1.4 Basic Security Analysis

Before delving into the security of the *Eagle** encryption algorithm, we first introduce another temporary encryption algorithms, named A-Encrypt algorithm.

The encryption process of the A-Encrypt algorithm is as follows

Algorithm 11 A-Encrypt.

Input:Key: (δ) .Plaintext sequences: $R = (R_1, R_2, \dots, R_K)$.**Output:**Ciphertext sequences: $C = (S_1, S_2, \dots, S_K)$.1: **for** $i = 1; i \leq K; i++$ **do**2: $S_i \leftarrow R_i * \delta * \delta$ 3: **end for**

The decryption process of the A-Encrypt algorithm is as follows

Algorithm 12 A-Decrypt.

Input:Key: (δ) .Ciphertext sequences: $C = (S_1, S_2, \dots, S_K)$.**Output:**Plaintext sequences: $R = (R_1, R_2, \dots, R_K)$.1: **for** $i = 1; i \leq K; i++$ **do**2: $R_i \leftarrow S_i / \delta / \delta$ 3: **end for**

Observing the plaintext and ciphertext structure of the *Eagle** encryption algorithm, we can easily find the following conclusions.

[**Theorem 13**] In the *Eagle** encryption algorithm, when know the plaintext $P = (P_1, P_2, \dots, P_K)$ and the corresponding ciphertext $C = (C_1, C_2, \dots, C_{K+1}, Sx_1, Sx_2, \dots, Sx_{K+1}, Sy_1, Sy_2, \dots, Sy_{K+1}, Sz_1, Sz_2, \dots, Sz_K, Su, Sv, P_{K+1})$, the problem of cracking its key (δ_1, δ_2) is equivalent to cracking two respective keys δ_1 and δ_2 under the condition of knowing only the ciphertext $(Sx_1, Sx_2, \dots, Sx_{K+1}, Su)$ of the first A-Encrypt algorithm and only the ciphertext $(Sy_1, Sy_2, \dots, Sy_{K+1}, Sv)$ of the second A-Encrypt algorithm.

Proof. According to the definition of *Eagle**, when know the plaintext $P = (P_1, P_2, \dots, P_K)$ and the corresponding ciphertext $C = (C_1, C_2, \dots, C_{K+1}, Sx_1, Sx_2, \dots, Sx_{K+1}, Sy_1, Sy_2, \dots, Sy_{K+1}, Sz_1, Sz_2, \dots, Sz_K, Su, Sv, P_{K+1})$, the problem of cracking its key (δ_1, δ_2) is equivalent to solve the following system of equations.

$$\left\{ \begin{array}{l} (P_1 \oplus Sz_1, *) \otimes (x_1, y_1) = (C_1, *) \\ (P_2 \oplus Sz_2, *) \otimes (x_2, y_2) = (C_2, *) \\ \dots\dots\dots \\ (P_K \oplus Sz_K, *) \otimes (x_K, y_K) = (C_K, *) \\ (P_{K+1}, *) \otimes (x_{K+1}, y_{K+1}) = (C_{K+1}, *) \end{array} \right. \quad (6.1)$$

$$\left\{ \begin{array}{l} x_1 * \delta_1 * \delta_1 = Sx_1 \\ x_2 * \delta_1 * \delta_1 = Sx_2 \\ \dots\dots\dots \\ x_{K+1} * \delta_1 * \delta_1 = Sx_{K+1} \\ u * \delta_1 * \delta_1 = Su \end{array} \right. \quad (6.2)$$

$$\left\{ \begin{array}{l} y_1 * \delta_2 * \delta_2 = Sy_1 \\ y_2 * \delta_2 * \delta_2 = Sy_2 \\ \dots\dots\dots \\ y_{K+1} * \delta_2 * \delta_2 = Sy_{K+1} \\ v * \delta_2 * \delta_2 = Sv \end{array} \right. \quad (6.3)$$

Where $x_1, x_2, \dots, x_{K+1}, z, y_1, y_2, \dots, y_{K+1}, u, v, \delta_1, \delta_2$ are unknown variables to be solved. $P_1, P_2, \dots, P_K, Sx_1, Sx_2, \dots, Sx_{K+1}, Sy_1, Sy_2, \dots, Sy_{K+1}, Sz_1, Sz_2, \dots, Sz_K, Su, Sv, P_{K+1}$ are known variables.

According to the conclusion of Theorem 9, each equation in the system of equations (6.1) can be regarded as an identity equation.

The unknown variables $x_1, x_2, \dots, x_{K+1}, u$ in the system of equations (6.2) can be equivalently regarded as the plaintexts of the A-encryption algorithm, with the known variables $Sx_1, Sx_2, \dots, Sx_{K+1}, Su$ being the ciphertexts of the A-encryption algorithm, and δ_1 being the key. Solving the system of equations (6.2) is equivalent to crack the key δ_1 under the condition of knowing only the ciphertext $(Sx_1, Sx_2, \dots, Sx_{K+1}, Su)$ of A-Encrypt algorithm.

Similarly, solving the system of equations (6.3) is equivalent to crack the key δ_2 under the condition of knowing only the ciphertext $(Sy_1, Sy_2, \dots, Sy_{K+1}, Sv)$ of the second A-Encrypt algorithm.

□

Normally, for a short key block encryption algorithm, when only the ciphertext is known, cracking possible keys can only be done using exhaustive search, but this is not necessarily the case. In fact, if an encryption algorithm only knows the ciphertext and meets the following three conditions, it can only use exhaustive search to crack possible keys.

- (i) Given any key, any ciphertext in one group is equally possible.
- (ii) Given any key, any plaintext in one group is equally possible.
- (iii) In the case of knowing only ciphertext, any key for one group is equally possible.

According to the the Algorithm 11 and Algorithm 12, A-Encrypt obviously

satisfies the above three conditions. This means that A-Encrypt can only use exhaustive search to crack the key when only knowing their ciphertexts, and there are no effective algorithms.

Next, consider several possible exhaustive attack methods.

[Case 1] Only enumerate δ_1 .

When δ_1 is exhausted, $x_1, x_2, \dots, x_{K+1}, u$ can be directly calculated from the system of equations (6.2), the system of equations (6.1) can be expressed as a system of equations about y_i and v .

$$\begin{cases} (\mathbb{E}_{u,v}(\mathbb{E}_{x_1,y_1}(P_1 \oplus Sz_1)) * (x_2 \oplus y_2)) \oplus \mathbb{E}_{x_2,y_2}(P_2 \oplus Sz_2) = C_2 \\ (\mathbb{E}_{u,v}(\mathbb{E}_{x_2,y_2}(P_2 \oplus Sz_2)) * (x_3 \oplus y_3)) \oplus \mathbb{E}_{x_3,y_3}(P_3 \oplus Sz_3) = C_3 \\ \dots\dots\dots \\ (\mathbb{E}_{u,v}(\mathbb{E}_{x_K,y_K}(P_K \oplus Sz_K)) * (x_{K+1} \oplus y_{K+1})) \oplus \\ \mathbb{E}_{x_{K+1},y_{K+1}}(P_{K+1} \oplus Sz_{K+1}) = C_{K+1} \end{cases} \quad (6.4)$$

Where $(y_1, y_2, \dots, y_{K+1})$ and v are unknown variables to be solved. From the system of equations (6.3), to solve δ_2 , We must solve for at least one of $(y_1, y_2, \dots, y_{K+1}, v)$ from the system of equations (6.4). However from the system of equations (6.4), it can be concluded that any two variables in $(y_1, y_2, \dots, y_{K+1}, v)$ can be arbitrarily set to solve for the other variables in $(y_1, y_2, \dots, y_{K+1}, v)$, thus we can't solve δ_2 without enumerating it.

[Case 2] Only enumerate δ_2 .

When δ_2 is exhausted, $y_1, y_2, \dots, y_{K+1}, v$ can be directly calculated from the system of equations (6.2), the system of equations (6.1) can be expressed as a system of equations about x_i and u , which is the same as (6.4). In a similar way, we can't solve δ_1 without enumerating it.

From the above analysis, it can be seen that for any known plaintext-ciphertext correspondence, the correct key can only be found by exhaustively enumerating all (δ_1, δ_2) .

Next, we need to consider another possible attack method. The attacker finds the relationship between the ciphertext to be decrypted and the ciphertexts-plaintexts in the sample from a large number of plaintext-ciphertext pairs, and directly decrypts or partially decrypts without cracking the key. The premise for this attack method to be effective is that there is some statistical relationship or some specific linear relationship between the plaintext and the ciphertext.

Due to the complete randomness of x_i and y_i and z_i , there is no statistical relationship or any form of mathematical relationship between the plaintext P_i and Sx_i . Similarly, there is no statistical relationship between P_i and Sy_i . Furthermore, according to Theorem 9, P_i and C_i are also completely unrelated. In fact, if we disregard all M and C (in reality, all M and C are invisible without solving for x_i, y_i, u and v), the encryption process of *Eagle** is equivalent to one-time pad. An attacker cannot find any patterns between plaintexts and ciphertexts from a large sample of plaintext-ciphertext pairs.

After a simple deduction, we can obtain the following expression among plaintext, key, and ciphertext.

$$\begin{aligned}
& (P_{i+1} \oplus Sz_{i+1}) \\
&= ((P_i \oplus Sz_i) * (Sx_i/\delta_1^2 \oplus Sy_i/\delta_2^2) * (Su/\delta_1^2 \oplus Sv/\delta_2^2)) \\
&\quad \oplus ((Sx_i/\delta_1^2)^{+(L-1)} * (Su/\delta_1^2 \oplus Sv/\delta_2^2)) \\
&\quad \oplus (((Sx_{i+1}/\delta_1^2)^{+(L-1)} \oplus C_{i+1})/(Sx_{i+1}/\delta_1^2 \oplus Sy_{i+1}/\delta_2^2)) \\
&\quad \oplus (Su/\delta_1^2)^{+(L-1)}
\end{aligned}$$

After collecting a large number of plaintext-ciphertext pairs, the attacker cannot directly decrypt new ciphertexts without cracking the key. This is because no matter what he does, he can only obtain the relationship between two adjacent pairs of plaintexts P_{i+1} and P_i , and the parameters dependent on this relationship cannot be obtained solely based on the ciphertexts.

6.1.5 Linear attack analysis to *Eagle** encryption algorithm

Linear attack is a very effective attack method proposed by M. Matsui[2] at the European Cryptology Conference in 1993. Later, scholars quickly discovered that the linear attacks are applicable to almost all block encryption algorithms, and linear attacks have become the main attacks for block encryption algorithms. Various new attacks based on linear attacks are constantly being proposed.

The core idea of linear attack is to take the nonlinear transformation in the cryptographic algorithm, such as the linear approximation of the S-box, and then extend the linear approximation to the linear approximation of the round function, and then connect these linear approximations to obtain a linear approximation of the entire cryptographic algorithm, finally a large number of known plaintext-ciphertext pairs encrypted with the same key are used to exhaustively obtain the plaintext and even the key.

We have noticed that the reason why linear attacks have become an effective attack for block encryption algorithms is that when the key is known, there is a certain implicit linear relationship between the ciphertext and the plaintext. By analyzing the known plaintext-ciphertext pairs, some effective linear relations can be obtained, and some bits of the key can be guessed.

In the *Eagle** encryption algorithm, the entire encryption system is divided into three independent parts, each with its own independent key and parameters. The ciphertext of the encryption system that includes the original plaintext is generated by mixing the original plaintext with random parameters, without knowing the random parameters, ciphertext can be regarded as completely independent of the original plaintext, which means that with random parameter changes, ciphertext can equally probability traverse any text in the ciphertext space. And the two other parts of the ciphertext is completely generated by random parameters, completely unrelated to the original plaintext. This can be

expressed as

$$\begin{aligned}
Pr(C_i = c_i | P_i = p_i) &= \frac{1}{2^{L-1}} \\
Pr(Sx_i = sx_i | P_i = p_i) &= \frac{1}{2^{L-1}} \\
Pr(Sy_i = sy_i | P_i = p_i) &= \frac{1}{2^{L-1}} \\
Pr(Sz_i = sz_i | P_i = p_i) &= \frac{1}{2^L} \\
Pr(Su = su | P_i = p_i) &= \frac{1}{2^{L-1}} \\
Pr(Sv = sv | P_i = p_i) &= \frac{1}{2^{L-1}} \\
Pr(P_{K+1} = p_{K+1} | P_i = p_i) &= \frac{1}{2^{L-1}}
\end{aligned}$$

6.1.6 Differential attack analysis to *Eagle** encryption algorithm

Differential attack was proposed by Biham and Shamir[6] in 1993, it is a chosen-plaintext attack. Its core idea is to obtain key information by analyzing specific plaintext and ciphertext differences.

The essence of a differential attack is to track the “difference” of the plaintext pair, where the “difference” is defined by the attacker according to the target, which can be an exclusive XOR operation or other target values.

For the *Eagle** encryption algorithm, suppose the differential attacker chooses two specific plaintexts P_1 and P_2 , their difference is Δ , that is $P_2 = P_1 + \Delta$, Since the ciphertexts in the second part and the third part are completely unrelated to the plaintext, only the ciphertext in the first part will be analyzed here, the corresponding ciphertexts are C_1 and C_2 , and the difference between the ciphertexts is ε as $C_2 = C_1 + \varepsilon$. Since C_1 and C_2 are completely random, it is uncertain whether the difference ε of the ciphertext is caused by randomness of (x_1, y_1, x_2, y_2, u) or the uncertainty of the plaintext (P_1, P_2) . Furthermore, for any key (δ_1, δ_2) , $x_1 * \delta_1 * \delta_1$, $y_1 * \delta_2 * \delta_2$ subject to the uniform distribution, which can be denoted as

$$\begin{aligned}
&Pr(C_1 = c_1, \dots, C_{K+1} = c_{K+1} | \\
&(P_1 = p_1, \dots, P_K = p_K, W = (\delta_1, \delta_2)) \\
&= 1/2^{(K+1)(L-1)} \\
&Pr(Sx_1 = sx_1, \dots, Sx_{K+1} = sx_{K+1} | \\
&(P_1 = p_1, \dots, P_K = p_K, W = (\delta_1, \delta_2)) \\
&= 1/2^{(K+1)(L-1)}
\end{aligned}$$

$$\begin{aligned} & Pr(Sy_1 = sy_1, \dots, Sy_{K+1} = sy_{K+1} | \\ & (P_1 = p_1, \dots, P_K = p_K, W = (\delta_1, \delta_2)) \\ & = 1/2^{(K+1)(L-1)} \end{aligned}$$

$$\begin{aligned} & Pr(Sz_1 = sz_1, \dots, Sz_K = sz_K | \\ & (P_1 = p_1, \dots, P_K = p_K, W = (\delta_1, \delta_2)) \\ & = 1/2^{KL} \end{aligned}$$

$$\begin{aligned} & Pr(Su = su, Sv = sv, P_{K+1} = p_{K+1} | \\ & (P_1 = p_1, \dots, P_K = p_K, W = (\delta_1, \delta_2)) \\ & = 1/2^{3(L-1)} \end{aligned}$$

That is to say, for any specific plaintext P_1 and P_2 selected by the attacker, after being encrypted with the same key, the corresponding block ciphertexts $(C_1, C_2, \dots, C_{K+1})$, $(Sx_1, Sx_2, \dots, Sx_{K+1})$, $(Sy_1, Sy_2, \dots, Sy_{K+1})$, $(Sz_1, Sz_2, \dots, Sz_K)$, Su, Sv, P_{K+1} are completely random, and every possible value in the ciphertext space appears with equal probability. The attacker has no way to capture the propagation characteristics of the “difference” in the plaintext.

References

- [1] Shannon C E. Communication theory of secrecy systems. *Bell System Technical Journal.*, 28(4):656–715, 1949.
- [2] M. Matsui. Linear cryptanalysis method for DES cipher. *Advances in cryptology: EUROCRYPT’93, LNCS765, springer verlag*, 1994:386-397.
- [3] Kaliski B S, Robshaw M J B. Linear Cryptanalysis Using Multiple Approximations. *Annual International Cryptology Conference. Springer, Berlin, Heidelberg*, 1994.
- [4] Biryukov, A., De Cannière, C., Quisquater, M. On Multiple Linear Approximations. In: Franklin, M. (ed.) CRYPTO 2004. LNCS, vol. 3152. Springer, Heidelberg, 2004.
- [5] Cho, J.Y., Hermelin, M., Nyberg, K. A new technique for multidimensional linear cryptanalysis with applications on reduced round serpent. In: Lee, P.J., Cheon, J.H. (eds.) ICISC 2008. LNCS, vol. 5461, pp. 383–398.
- [6] Eli Biham, Adi Shamir. Differential Cryptanalysis of the Data Encryption Standard. Springer-Verlag, 1993.
- [7] Biham E, Biryukov A, Shamir A. Cryptanalysis of Skipjack reduced to 31 rounds using impossible differentials. In: Advances in Cryptology—EUROCRYPT 1999. Springer Berlin Heidelberg, 1999: 12–23.

- [8] Tsunoo Y, Tsujihara E, Shigeri M, et al. Cryptanalysis of CLEFIA using multiple impossible differentials. In: 2008 International Symposium on Information Theory and Its Applications—ISITA 2008. pp. 1–6.
- [9] Sanjeev Arora, Boaz Barak. Computational Complexity: A Modern Approach. 2009.
- [10] Cook, Stephen A. The complexity of theorem-proving procedures. [J].ACM, 1971:151-158.